

Task/Criterion (Note: each level reached if conditions met)	Preconditions	None	Unacceptable	Poor	Fair	Good	Excellent
Task 1: Construct code (25)		(0) Not attempted	(5) The code shows some effort, but it mostly does not match the provided and class diagram OR a considerable amount of it was not written (i.e. it covers few use cases).	(10) The code does not correctly (i.e. meeting requirements) cover $\geq$ half of the required use cases OR it covers more (not all) but quality* is very low (i.e. unreadable), even if those that it covers do mostly respect the class and the sequence diagrams.	(15) The code covers the use cases and mostly meets requirements. It matches the class diagram or explanations are provided for very small differences to them. Reasonable attempt at implementing "Pre-register faculty member" use case. Quality could be improved in some important aspects OR insufficient effort put into catching invalid inputs using assertions.	(20) Like 'Fair' but only small aspects of code quality could be improved. Good attempt at implementing "Pre-register faculty member" use case, including Javadoc. Reasonable catching of invalid inputs using assertions.	(25) Like 'Good' but code fully meets requirements, of very high quality and impresses by extra effort in doing a lot of input validation OR considering non-functional requirements/attributes OR other aspects going beyond what was required.
Task 2: Unit testing (15)	At least "Poor" in Task 1 and having reasonably	(0) Not attempted	(3) Some effort but not understanding what unit tests should do.	(6) $\geq$ half of the required classes or $\geq$ half of their methods not having unit	(9) Unit tests for all required classes and for (almost) all their methods are	(12) Unit tests for all required classes and their methods are provided	(15) Unit tests for all required classes and their methods are provided and pass, and testing a high number

* As described in Lecture 14.

	implemented the required classes.			tests OR testing only one set of inputs (in one test) for each method OR most tests don't pass.	provided, and there is some effort to test a few classes of inputs for each method, however the structuring of the tests, naming, comments, messages OR use of assert methods could be improved. Most tests pass.	and pass, and good effort at testing several classes of inputs for each method. Structuring of the tests, naming, comments, messages and use of assert methods usually good with small issues.	of classes of inputs for each method. High coverage (>70%) of all the classes being tested. Structuring, naming, comments, messages and use of assert all excellent.
Task 3: Create system-level tests (20)	At least "Poor" in Task 1	(0) Not attempted	(4) Some effort but not understanding what system tests should do.	(8) $\geq$ half of the use cases don't have system tests OR only testing one scenario per use case OR most tests don't pass.	(12) System tests for all use cases are provided, and there is some effort to test a few scenarios for each use case. The structuring of the tests, naming, comments, messages OR use of assert methods could be improved. Most tests pass.	(16) System tests for all use cases are provided and pass, and good effort at testing several scenarios for each use case. Structuring of the tests, naming, comments, messages and use of assert methods usually good	(20) System tests for all use cases are provided and pass, and a high number of scenarios for each use case are tested. Structuring, naming, comments, messages and use of assert all excellent.

* As described in Lecture 14.

						with small issues.	
Task 4: Code review (10)		(0) Not attempted	(2) Hardly any comments submitted.	(4) Brief review which does not respect many of the good review guidance OR which has missed on providing many possible comments.	(6) Reasonable review which missed some of the good review guidance OR missed on providing some possible comments.	(8) Good review respecting the good review guidance. Few minor comments missed.	(10) Excellent effort at providing a thorough review, fully respecting the good review guidance and providing truly useful comments.
Task 6: Reflection (15)	At least 'Poor' in Tasks 1, 2, 3.	(0) Not attempted	(3) Little use of reflection – mostly broad unjustified, even dishonest statements	(6) Some use of reflection but generally too brief or only on team work/work.	(9) An attempt to use the given reflective model or similar on a few of the suggested topics for team work, or to touch on a few aspects of the work, or more but just in one. Min. half a page.	(12) A good attempt to use the given reflective model or similar on a good number of the topics for team work and work. Some discussion of tools for team work, the marking scheme, quality attributes, experience with approaches/ tools/ agile practices with	(15) An excellent attempt showing a highly reflective team, touching on most of the proposed topics, the marking scheme, and a good number of approaches/ tools/ agile practices with choices appropriate, good understanding and proof of any tools used (screenshots). Would normally expect around a one and a half pages.

* As described in Lecture 14.

						choices appropriate, good understanding and proof of any tools used (screenshots). Min. a page.	
Exceptionality	15 marks that can be awarded at the discretion of the marker if mark of >80 (max=85) reached for exceptional programming and testing skills, exceptional use of approaches/ tools/ agile practices, exceptional team work.						

* As described in Lecture 14.